

DPI343

Computer Organization and Assembly Language

Lecture 10.1

Real address mode in 16-bit environment

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

16-bit environment and Real address mode

- The second part of the COAL course covers interrupt handling and the principles of operation of various computer components, **so we will need to access memory addresses and I/O ports that are not normally accessible**
- **CPU starts in real mode on reset**, but operating systems switch CPU to protected mode at startup and normally never return to real mode
- All **modern operating systems** (Windows, Linux, etc.) **run in Protected mode** to have control over application software and increase security
- **Real address mode does not provide memory protection**, so we will be able to access memory areas that are not available when working in modern operating systems
- Access to all addressable memory, I/O ports and hardware is possible in Real address mode
- **Our programs to process hardware interrupts and access to computer components will be created for the 16-bit environment in Real address mode**
- **Programs written for Real mode cannot run in Protected mode** therefore our experimental programs will not be able to run directly in a Windows environment
- We'll use emulation environment DOSBox and **execute Real mode programmes only in the emulation environment**

I/O ports (a brief overview)

- CPU can also transfer data to and from input/output ports
- I/O ports are created in system hardware by circuitry that decodes the control, data, and address pins on the processor
- An I/O port can be an input only port, an output only port, or a bidirectional port
- **Some I/O ports are used for transmitting data, other I/O ports are used to control peripheral devices**
- An I/O port is a device that to the computer looks like a memory cell but contains connections to the «outside world»
- Assembly instructions **in** and **out** are used to read or write information from ports:
 in accum, port
 out port, accum
 accum – AL, AX or EAX
 port – number of the port (immediate value or in DX)

In the next topics, we'll use ports to work with different computer components in Real address mode

BIOS Data Area

The BIOS Data Area (BDA) is part of memory starting from address 0000:0400 which stores different values used by BIOS.

<u>Address</u>	<u>Content</u>
0000:0417	2 bytes - keyboard status bits
0000:041E	32 bytes - keyboard buffer
0000:0449	1 byte - current video mode
0000:046C	4 bytes - time counter
etc.	

We'll use information from the BIOS Data Area to work with different computer components in Real address mode

COAL Toolkit for the 16-bit environment

Unzip COAL Toolkit for the 16-bit environment on your computer (file `_COAL2025 16.zip` on the course page)

The Toolkit for the 16-bit environment includes:

- the **MASM16_Tools** folder with
 - the Microsoft compiler **ml.exe** and link editor **link16.exe** for the 16-bit environment
 - the **emulation environment DOSBox** tools
 - the debugger **afd.exe** for the 16 bit environment
- the **MyAsm16 folder**, which contains some source code examples for the 16-bit environment and where you can store your own developed program files
- **dosbox-0.74-3.conf** – a configuration file for the emulation environment
- **scripts** for more convenient work with development tools

Scripts

Scripts provide similar functionality to those that were available for the 32-bit environment:

- **masm_x86_16_cmd.bat** – environment configuration and opening of console window,
- **b16.bat** – builds the executable in .exe format,
- **emul.bat** – opens the DOSBox emulation environment.

A typical configuration of the emulation environment DOSBox is in the **dosbox-0.74-3.conf** file, but you can adjust options following recommendations given in file *Tips for customizing emulation environment.pdf*

You can also try to use 16-bit tools without these scripts (on your own risk)

How to build and run an executable file

- Prepare .asm file in your folder for a 16-bit applications using:
 - .186 as the CPU selection directive
 - .MODEL SMALL as the memory model directive
 - .STARTUP as the entry point directive
 - the .STARTUP directive generates start-up code for the given CPU type and the memory model defined by the .MODEL directive. It defines an entry point label, so that you don't need to write entry point address in the END directive, and it initializes segment registers*
 - only 16 bit registers (ax, bx, cx, dx, si, di etc.) for operations
 - directive .EXIT to return from your application to operating system (in fact, to the simulation environment)
- Compile .asm file: **`m1 /c filename.asm`**
- Build executable in .exe format from the .obj file: **`link16 filename,,,,,`** 
- Open 16-bit simulation environment (make sure your disk folder for 16-bit applications is mounted properly and is a current folder)
- Run your executable code *filename.exe* in the simulation environment window
- or
- Run it in the debugger: *afd filename* which can be used only in the simulation environment

More convenient way is to use ready-made scripts available in the COAL Toolkit

Example hello.asm

The CS register points to the code segment, DS and ES to data segment, SS to stack segment.
The stack is placed in the high addresses of the SS segment.

```
.186
.model small
option casemap:none
```

```
.stack
```

```
.data
msg byte "Hi! This is executable code in .EXE format for 16-bit environment",13,10,'$'
```

```
.code
.startup                ; directive to prepare segment addressing
main proc
```

```
    mov dx, offset msg   ; parameter for the system call - address of the message
    mov ah, 9            ; parameter for the system call - function 9 - console output
    int 21h              ; software interrupt to initiate the system call
```

```
.exit 0                 ; directive to return to the operating system
main endp
```

```
end
```

Build and execute

Compile and link:

```
ml /c hello.asm  
link16 hello,,,,,
```

Open emulation environment (i.e. run the script emul.bat):

```
emul
```

and run hello.exe in the emulation window

Or use COAL Toolkit script b16.bat:

Build 16-bit executable (compile and link):

```
b16 hello
```

Open emulation environment:

```
emul
```

and run hello.exe in the emulation window

Real Address Mode (8086)

Addresses in real mode always correspond to real locations in memory.

Memory address consists of segment and offset components:

segment : offset

Segment value is in one of the 16-bit segment registers (CS, DS, etc.)

Offset value can be an immediate value or in the general purpose register

One segment in real address mode takes up to 64 KB (because max offset value is $2^{16}-1$)

Segment always begins at paragraph memory address boundary.

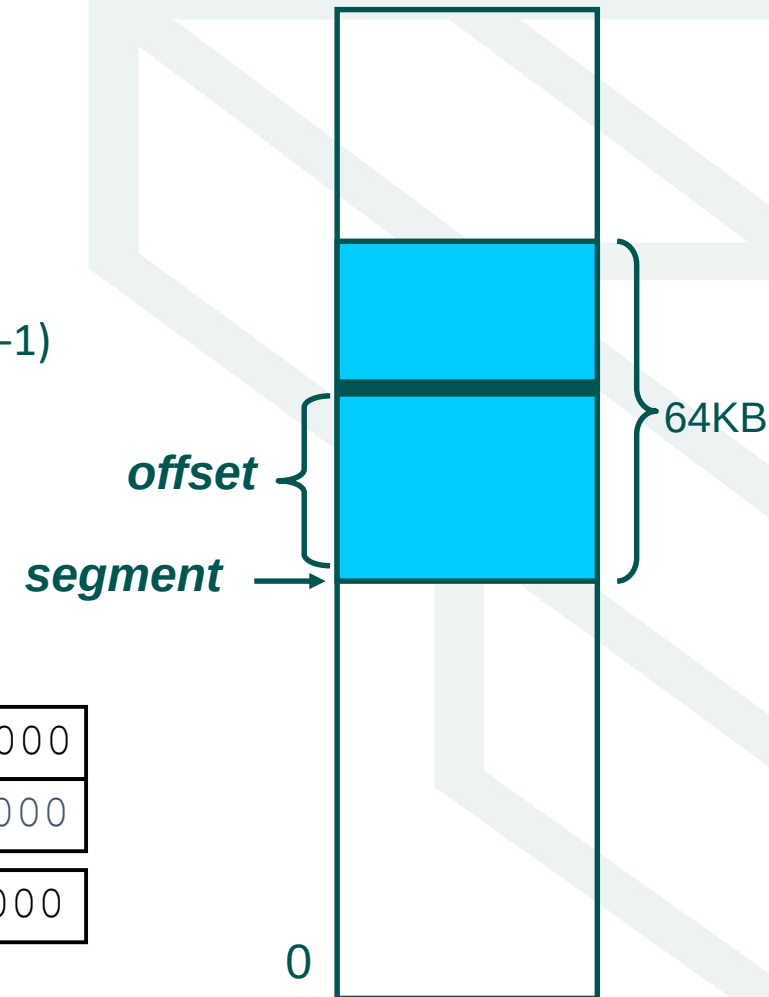
Multiple segments can overlap.

Physical address (sometimes called Effective address) is calculated from segment and offset values

Physical address = *segment* * 16 + *offset*

segment	0000000001000000	0000
+ offset		00000000000011000
Physical address	00000000010000011000	

So 2^{20} bytes can be addressed (1 MB)



We'll use only 16-bit General Purpose Registers

15	8 7	0	16-bit
AH	AL		AX
BH	BL		BX
CH	CL		CX
DH	DL		DX
BP			
SI			
DI			
SP			

Some simple I/O functions for 16-bit environment

- Load the AH register with a function number
- Load other parameters in registers specific for particular function
- Call function via interrupt instruction *int 21h* (software interrupt 21h)
- Receive return values via registers

Display text string

ah = 9

ds:dx = address of the text

Text should be terminated with '\$'.

In most cases the data segment register DS already contains segment value so you have to load only DX with the offset value.

```
message1    byte    'Please enter your name:$'
            ...
            mov  ah,9
            mov  dx, offset message1
            int  21h
```

Some simple I/O functions for 16-bit environment (cont.)

Buffered Keyboard Input

ah=0Ah

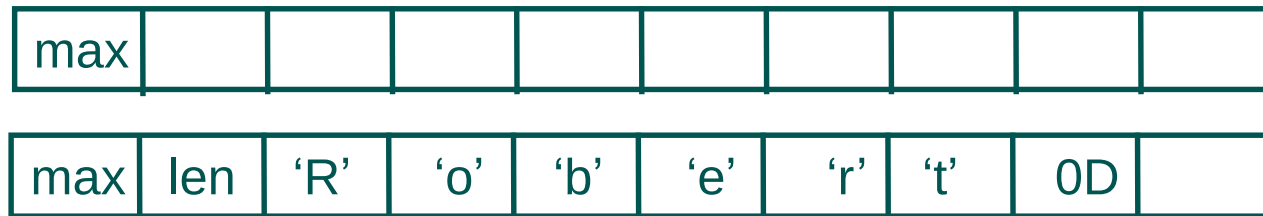
ds:dx = address of the input buffer

Result: line of text from the keyboard stored into the input buffer with a terminal CR symbol.

Before execution of the instruction int 21h the first byte of the buffer must contain the maximum number of characters expected in the input buffer.

Actual number of characters entered will be stored in the second byte after execution.

The actual input characters begin at the third byte of the buffer.



```
buff    byte    7, 0, 8 dup(0)
        ...
        mov ah, 0Ah
        mov dx, offset buf
        int 21h           ; text starts from buf+2 !
```